```
stoplight_color = %w[ green red yellow ].sample
expect(stoplight_color).to eq('green').or eq('red').or eq('yellow')
```

As we see in the stoplight_color example, you can string matchers together into arbitrarily long chains. You can use the words and/or, or you can use the & and | operators:

10-exploring-rspec-expectations/04/composing_matchers.rb

```
alphabet = ('a'..'z').to_a
expect(alphabet).to start_with('a') & end_with('z')

stoplight_color = %w[ green red yellow ].sample
expect(stoplight_color).to eq('green') | eq('red') | eq('yellow')
```

This syntax might seem really fancy and complex, but internally, it's quite simple. These methods return a new matcher that wraps the two operands. The and matcher only succeeds if both of its operands match. The or matcher succeeds if either operand matches.

Poking around in IRB will give you a sense of how these work:

```
>> start_with_a_and_end_with_z = start_with('a').and end_with('z')
=> #<RSpec::Matchers::BuiltIn::Compound::And:0x007f94dc83ba30
    @matcher_1=#<RSpec::Matchers::BuiltIn::StartWith:0x007f94dc82bd38
                @actual_does_not_have_ordered_elements=false, @expected="a">,
    @matcher_2=#<RSpec::Matchers::BuiltIn::EndWith:0x007f94dc82bc20
                @actual_does_not_have_ordered_elements=false, @expected="z">>
```

Here we've created a compound matcher, an instance of RSpec::Matchers::BuiltIn::Compound::And, which keeps internal references to the original start_with and end_with matchers.

The new compound matcher works just like any other. It even provides its own error message, based on the messages of the underlying matchers:

```
>> expect(['a', 'z']).to start_with_a_and_end_with_z
=> true
>> expect(['a', 'y']).to start_with_a_and_end_with_z
RSpec::Expectations::ExpectationNotMetError: expected ["a", "y"] to end ↩
with "z"
    « backtrace truncated »
>> expect(['b', 'y']).to start_with_a_and_end_with_z
RSpec::Expectations::ExpectationNotMetError:    expected ["b", "y"] to ↩
start with "a"

...and:

   expected ["b", "y"] to end with "z"
    « backtrace truncated »
```

Compound matchers are smart enough to show failure messages only for the bits that failed.